

4. Écrire à plusieurs endroits

La position est un calcul comme un autre.

1. Ce qu'il faut comprendre

Trois appels, trois endroits. Rien de neuf — sauf ceci : la colonne et la ligne **ne sont pas obligées d'être écrites en clair**.

`texte(4 + 2, 8, "...")` marche, et `texte(x, y, "...")` avec des variables aussi. C'est ce qui rendra possible tout ce qui bouge.

2. Le programme, morceau par morceau

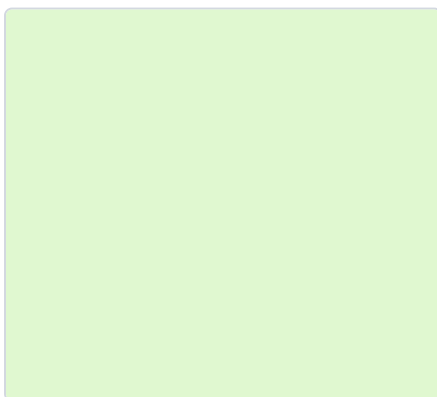
Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1342 octets** de cartouche.

1. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

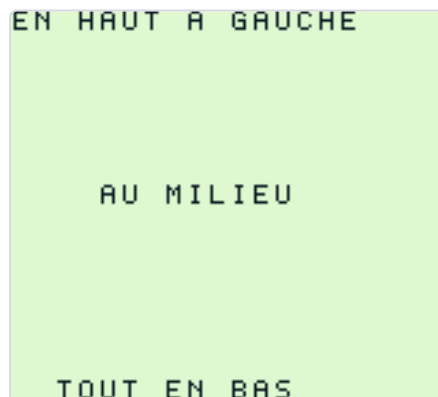
```
int main() {  
    texte(0, 0, "EN HAUT A GAUCHE");  
    texte(4, 8, "AU MILIEU");  
    texte(2, 17, "TOUT EN BAS");  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Trois lignes, en haut, au milieu et tout en bas.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la première touche le coin
 - ✓ la deuxième est au milieu
 - ✓ la troisième est sur la dernière ligne
-

Ensuite : **5. Une constante plutôt qu'un nombre** — niveau 1 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++